

Singleton Design Pattern

Introduction

The **Singleton design pattern** ensures that **only one instance** of a class exists throughout the application lifecycle and provides a **global point of access** to it.

- **Category:** Creational Design Pattern
 - **Core idea:** *One class, one instance, one access point.*
-

Why Singleton Came Into Existence?

Before Singleton: - Multiple uncontrolled instances of critical classes (e.g., loggers, configuration managers). - Inconsistent state when multiple objects represented the same concept. - High memory usage by creating unnecessary duplicate objects.

Singleton solves this by: - Restricting object creation. - Maintaining one shared instance. - Ensuring consistency and saving memory/resources.

Structure

1. **Private constructor** → prevents direct instantiation.
 2. **Static instance** → stores the single object.
 3. **Public access method** → provides global access.
-

Implementations

1. Eager Initialization

```
public class Singleton {  
    private static final Singleton instance = new Singleton();  
  
    private Singleton() { }  
  
    public static Singleton getInstance() {  
        return instance;  
    }  
}
```

-  Thread-safe, simple -  Wastes memory if not used
-

2. Lazy Initialization

```
public class Singleton {
    private static Singleton instance;

    private Singleton() { }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

-  Creates instance only when needed -  Not thread-safe

3. Thread-Safe (Synchronized)

```
public class Singleton {
    private static Singleton instance;

    private Singleton() { }

    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

-  Safe for multithreading -  Slower due to synchronization overhead

4. Double-Checked Locking

```
public class Singleton {
    private static volatile Singleton instance;

    private Singleton() { }

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
```

```

        instance = new Singleton();
    }
}
return instance;
}
}

```

- ☒ Thread-safe, efficient - ☒ Synchronization only on first creation

5. Bill Pugh Singleton

```

public class Singleton {
    private Singleton() { }

    private static class SingletonHelper {
        private static final Singleton INSTANCE = new Singleton();
    }

    public static Singleton getInstance() {
        return SingletonHelper.INSTANCE;
    }
}

```

- ☒ Thread-safe - ☒ Lazy-loaded - ☒ No synchronization overhead - 9 Recommended in Java

6. Enum Singleton

```

public enum Singleton {
    INSTANCE;

    public void doSomething() {
        System.out.println("Doing something...");
    }
}

```

- ☒ Simplest, thread-safe by default - ☒ Serialization-safe - ☒ Less flexible (no lazy init, no subclassing)

Why Singleton is Better

- **Consistency:** One shared object across system
- **Resource Saving:** Prevents multiple heavy object creation
- **Simplified Access:** Global access point
- **Thread Safety** (with correct implementation)

When to Use

✓ Use Singleton when: - Exactly **one instance** is required - Shared resources are needed (logger, configuration, DB connection pool) - Centralized control (thread pool, cache manager)

Examples: - Logger (`Log4j`) - Configuration Manager - Database Connection Pool - Cache Manager - `Runtime.getRuntime()` in Java

When NOT to Use

✗ Avoid Singleton when: - **Unit Testing** → Hard to mock due to hidden dependencies - **Scalability** → Single instance may become bottleneck in distributed systems - **Global State Issues** → Can lead to bugs and debugging difficulty - **Overuse** → Overusing Singletons often leads to anti-patterns

Applications

- **Java Runtime** → `Runtime.getRuntime()`
 - **Logger Frameworks** → Log4j, SLF4J
 - **Spring Beans** → Default scope is Singleton
 - **JDBC DriverManager** → Manages database drivers
 - **AWT Desktop** → `java.awt.Desktop.getDesktop()`
-

Comparison with Other Creational Patterns

Feature / Pattern	Singleton	Factory	Abstract Factory	Builder	Prototype
Purpose	One global instance	Creates objects without exposing logic	Creates families of related objects	Step-by-step object construction	Clone existing objects
Instances	Exactly one	Many	Many	Many	Many (copies)
Focus	Single access	Encapsulation of object creation	Encapsulation of multiple factories	Construction process	Copying an existing object

Feature / Pattern	Singleton	Factory	Abstract Factory	Builder	Prototype
Usage	Shared resources	When object creation is complex	When products must be consistent across families	When object has many optional params	When cloning is cheaper than creating
Example	Logger, Config	<code>Calendar.getInstance()</code>	GUI themes, toolkits	StringBuilder	Prototype shapes

Summary

The **Singleton design pattern** ensures that a class has only one instance with a global access point. It is extremely useful for **shared resources** like loggers, configuration managers, and DB connections. However, overusing it can lead to tight coupling, testability issues, and scalability problems.